

Temas Selectos de Física Computacional III:

Introducción a la Ciencia de datos.

Clase 6: 23 de septiembre de 2026

Dr. Jaime Octavio Guerra Pulido

1 Introducción

En esta clase se estudiará el problema de dimensionalidad y el método de componentes principales aplicado al conjunto de datos de cáncer de mama.

2 Mejorando la predicción de cáncer

En la clase pasada y en la tarea, se estudió la correlación de todas las variables con el diagnóstico (**target**) y, se observó que, como en muchos problemas de la vida real, no siempre hay una variable individualmente fuerte para separar entre clases. También puede ser que las características estén correlacionadas, lo que significa que contienen información redundante. Además, algunas variables pueden contener información complementaria que una sola no captura. Por lo tanto, **se debe mejorar el modelo** probando otras técnicas, ajustando los umbrales o probando un modelo de clasificación más avanzado.

3 Dimensionalidad

A medida que aumenta el número de variables o dimensiones en un conjunto de datos, la cantidad de datos necesarios para obtener un resultado estadísticamente significativo aumenta exponencialmente. Esto puede llevar a problemas como sobreajuste, aumento del tiempo de cálculo y reducción de la precisión de los modelos de aprendizaje automático. Este fenómeno se conoce como la **maldición de la dimensionalidad**, que se refiere a los problemas que surgen al trabajar con datos con muchas variables.

A medida que aumenta el número de dimensiones, el número de posibles combinaciones de variables de entrada aumenta exponencialmente, lo que dificulta obtener una muestra representativa de los datos. Realizar tareas como clasificación se vuelve más costoso porque los algoritmos necesitan procesar un espacio de características mucho más grande, lo que aumenta el tiempo de cómputo y la complejidad. Además, **algunos algoritmos de aprendizaje automático**

pueden ser sensibles al número de dimensiones requiriendo más datos para lograr el mismo nivel de precisión que con datos de menor dimensionalidad.

La **reducción de dimensionalidad** es un tipo de técnica con la que se pretende reducir el número de variables de entrada mientras se conserva la mayor cantidad posible de la información original.

4 Análisis de componentes principales (PCA)

El análisis de componentes principales (PCA) fue introducido por el matemático Karl Pearson en 1901. Funciona bajo la condición de que, al mapear los datos de un espacio de mayor dimensión a un espacio de menor dimensión, **la varianza de los datos en el espacio de menor dimensión debe ser máxima**.

El PCA es un procedimiento estadístico que utiliza una transformación ortogonal para convertir un conjunto de variables correlacionadas en un conjunto de variables no correlacionadas. Es la herramienta más utilizada en el análisis exploratorio de datos y en el aprendizaje automático para modelos predictivos.

El PCA se utiliza para reducir la dimensionalidad de un conjunto de datos al encontrar un nuevo conjunto de variables, más pequeño que el conjunto original, que retenga la mayor parte de la información de la muestra y sea útil para la regresión y clasificación de datos.

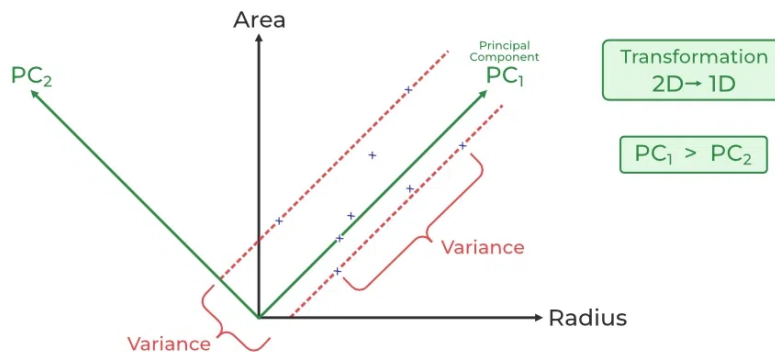


Figure 1: El método de PCA.

4.1 Estandarización

Es necesario estandarizar el conjunto de datos para asegurar que cada variable tenga una media de 0 y una desviación estándar de 1.

$$Z = \frac{X - \mu}{\sigma},$$

donde:

- X son las características independientes,
- μ es la media de las características independientes,
- σ es la desviación estándar de las características independientes.

4.2 Cálculo de la matriz de covariancia

La covariancia mide la fuerza de la variabilidad conjunta entre dos o más variables, indicando cuánto cambian en relación entre sí. La covarianza se puede calcular como:

$$\text{cov}(x_1, x_2) = \frac{1}{n-1} \sum_{i=1}^n (x_{1,i} - \bar{x}_1)(x_{2,i} - \bar{x}_2).$$

Los valores de la covarianza pueden ser:

- Positivos: A medida que x_1 aumenta, x_2 también aumenta.
- Negativos: A medida que x_1 aumenta, x_2 disminuye.
- Cero: No hay relación directa.

4.3 Cálculo de valores y vectores propios

A continuación, debemos calcular los valores propios (λ) y los vectores propios de la matriz de covarianza para identificar los componentes principales. Sea A una matriz cuadrada $n \times n$ y $\mathbf{X}$ un vector no nulo para el cual:

$$A\mathbf{v} = \lambda\mathbf{v}$$

donde λ es el valor propio y $\mathbf{v}$ es el vector propio correspondiente de la matriz A .

4.4 Selección de los componentes principales

Una vez que se tienen los valores propios, se seleccionan **los k componentes principales** que tienen los valores propios más altos. Estos componentes explican la mayor parte de la varianza en los datos.

4.5 Proyección de los datos

Proyectamos los datos originales en el espacio de los componentes principales para obtener una representación en dimensiones reducidas:

$$Y = \mathbf{v}W$$

donde $\mathbf{v}$ es la matriz original de datos y W es la matriz de vectores propios seleccionados.

5 Implementación del PCA

A continuación, se muestra cómo implementar el PCA paso a paso:

```
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer(as_frame=True)
df = cancer.frame
X = df[cancer['feature_names']]
```

El primer paso es estandarizar los datos. Para ello, se debe calcular la media y la desviación estándar de cada característica en el espacio de características.

```
X_mean = X.mean()
X_std = X.std()
Z = (X - X_mean) / X_std
```

La matriz de covarianza ayuda a visualizar la dependencia entre dos características en el espacio de características.

```
c = Z.cov()
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(12, 12))
sns.heatmap(c)
plt.show()
```

Ahora se calcularán los valores propios y los vectores propios para este espacio. Estos son cruciales para identificar los componentes principales de nuestro espacio de características.

```
import numpy as np
eigenvalues, eigenvectors = np.linalg.eig(c)
print('Valores propios:\n', eigenvalues)
print('Forma de los valores propios:', eigenvalues.shape)
print('Forma de los vectores propios:', eigenvectors.shape)
```

Los valores propios se deben ordenar de manera descendente y organizar los vectores propios correspondientes de manera adecuada.

```
idx = eigenvalues.argsort()[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
```

La **varianza explicada** es el término que nos da una idea de la cantidad de varianza total que ha sido retenida al seleccionar los componentes principales en lugar del espacio original de características.

```
explained_var = np.cumsum(eigenvalues) / np.sum(eigenvalues)
plt.plot(range(1, len(explained_var) + 1), explained_var)
plt.xlabel("Número de componentes")
plt.ylabel("Varianza explicada acumulada")
plt.grid()
plt.show()
```

Ahora vamos a determinar el número de componentes principales. Aquí consideramos una varianza explicada mayor o igual al 50%.

```
n_components = np.argmax(explained_var >= 0.50) + 1
```

Finalmente, falta proyectar los datos en los componentes principales seleccionados. La matriz de proyección es una matriz de vectores propios correspondientes a los valores propios más grandes de la matriz de covarianza de los datos.

```
import pandas as pd
u = eigenvectors[:, :n_components]
pca_component = pd.DataFrame(u,
                             index = cancer['feature_names'],
                             columns = ['PCA1', 'PCA2'])
plt.figure(figsize =(5, 7))
sns.heatmap(pca_component, annot=True, cmap='coolwarm')
plt.show()
```

Esta matriz nos dice qué variables originales son más relevantes en la nueva representación de PCA. Así, PCA1 y PCA2 forman un nuevo espacio de características donde podemos visualizar y clasificar mejor los tumores.

Finalmente, proyectamos nuestro conjunto de datos usando la fórmula:

$$P_i(u) = P_i \cdot u$$

La reducción de dimensionalidad se logra reteniendo solo aquellos ejes que explican la mayor parte de la varianza y descartando el resto.

En Python, necesitamos usar el producto matricial, denotado por @, que multiplica la matriz de datos estandarizada Z por la matriz de eigenvectores `pca_component`, lo que proyecta los datos originales en los nuevos componentes principales.

```

Z_pca = Z @ pca_component
Z_pca.columns = ['PCA1', 'PCA2']
print(Z_pca)

```

Los vectores propios de la matriz de covarianza de los datos se denominan **ejes principales** de los datos, y la proyección de las instancias de datos sobre estos ejes se denominan las **componentes principales**. La reducción de dimensionalidad se logra reteniendo solo aquellos ejes (dimensiones) que explican la mayor parte de la varianza.

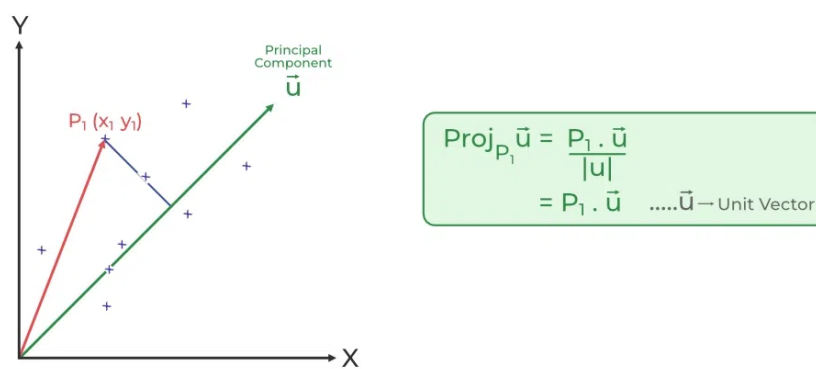


Figure 2: Ejes de PCA.

Los PCA1 y PCA2 son ortogonales y esto significa que su covarianza debe ser cero. Verifica lo anterior como:

```
print(Z_pca[['PCA1', 'PCA2']].corr())
```

Entonces, los tumores malignos y benignos se separan en el espacio de PCA:

```
Z_pca['target'] = df['target'].map({0: 'Maligno (0)', 1: 'Benigno (1)'})
```

```

plt.figure(figsize=(8,8))
sns.scatterplot(data=Z_pca, x="PCA1", y="PCA2", hue="target")
plt.xlabel('PCA1')
plt.ylabel('PCA2')
plt.show()

```

6 Resumen de PCA

Supongamos ahora que **no existiera ninguna variable** con correlación fuerte con el target. En ese caso:

- El PCA resulta útil porque permite proyectar el espacio de variables en direcciones que concentran la mayor parte de la varianza.
- Dichas proyecciones podrían revelar *estructuras latentes* en los datos (por ejemplo, agrupamientos) que no son evidentes en las variables originales.
- A partir de estas componentes, un clasificador supervisado podría encontrar fronteras de decisión más simples y efectivas que en el espacio original.
- Además, al reducir la dimensionalidad, se disminuye el ruido y la redundancia entre variables, lo cual puede mejorar la generalización del modelo.

7 Implementación del PCA en Scikit-learn al conjunto de Iris

Aplique el método de PCA al conjunto de datos de Iris para demostrar cómo se puede reducir de 4 dimensiones a 2, manteniendo la mayor parte de la información (porcentaje de la varianza total). Para cargar el conjunto en Scikit-learn, usa:

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
```

La implementación del PCA en Scikit-learn para solo 2 componentes:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```